1. Ir al directorio

2. Instalar dependencias

pip install -r requirements.txt

3. Ejecutar (la API key la pedirá si no está configurada)

Necesitarás una API Key gratuita de Google Gemini → aistudio.google.com/app/apikey

python chatbot.py

4. ¿Qué es StudyBot?

StudyBot es un chatbot de terminal que actúa como tu asistente personal de organización académica. Usa la IA de Google Gemini para responder preguntas sobre:

- Distribución de horas de estudio entre asignaturas
- Generación de horarios semanales personalizados
- Optimización del tiempo libre en fin de semana
- Técnicas de estudio y productividad

---

5. Arquitectura del script

Módulos principales

Clase / Función	Responsabilidad
StudentProfile	Modelo de datos del estudiante (asignaturas, horas, preferencias) con serialización JSON
ProfileSetup	Interfaz de terminal interactiva para crear/editar el perfil
StudyBot	Conecta con la API de Gemini, gestiona el historial de conversación y el system prompt
ChatUI	Loop principal de chat, renderizado visual con Rich, gestión de comandos
get_api_key()	Obtiene la clave API desde variable de entorno, archivo .env o la solicita al usuario
main()	Punto de entrada: orquesta el flujo completo

6. Cómo funciona la IA

System Prompt inteligente

Al iniciar, el bot inyecta **todo el perfil del estudiante** en el system prompt de Gemini:

PERFIL DEL ESTUDIANTE:

Nombre: Ana

Nivel: universidad

Priorizar ocio en fin de semana: Sí

ASIGNATURAS:

- Matemáticas (importancia: 5/5, dificultad: 4/5, créditos: 6.0)
- Historia (importancia: 3/5, dificultad: 2/5, créditos: 4.5)

...

HORAS DISPONIBLES:

- Lunes: 3.0h
- Martes: 3.0h

...

- Sábado: 0.5h ← mínimo por preferencia de ocio
- Domingo: 0.5h

Esto permite que **cada respuesta de Gemini** sea consciente del contexto del estudiante sin que tenga que repetirlo.

Algoritmo de distribución de horas

El bot calcula el peso de cada asignatura con la fórmula:

$$peso_i = importancia_i \times 0.6 + dificultad_i \times 0.4$$

Las horas por asignatura se distribuyen proporcionalmente:

$$horas_i = \frac{peso_i}{\sum pesos} \times horas_totales_semana$$

Modo ocio (fin de semana libre)

Cuando el estudiante activa priorizar_ocio, el system prompt instruye a Gemini a:

1. Reducir Sábado y Domingo a **0–1 hora máximo** (solo repaso ligero).
2. Compensar esas horas distribuyéndolas entre Lunes–Viernes.

7. Persistencia del perfil

El perfil se guarda automáticamente en student_profile.json en el mismo directorio:

json

```
{
  "nombre": "Ana",
  "nivel": "universidad",
  "asignaturas": [
    {"nombre": "Matemáticas", "importancia": 5, "dificultad": 4, "creditos": 6.0},
    {"nombre": "Historia",    "importancia": 3, "dificultad": 2, "creditos": 4.5}
  ],
  "horas_por_dia": {
    "Lunes": 3.0, "Martes": 3.0, "Miércoles": 2.5,
    "Jueves": 3.0, "Viernes": 2.0, "Sábado": 0.5, "Domingo": 0.5
  },
  "priorizar_ocio": true
}
```

Al relanzar el script, se carga automáticamente, evitando reconfigurar todo cada vez.

---

8. Interfaz visual (Rich)

El script usa la librería **Rich** para una terminal atractiva con:

- Paneles con bordes y colores diferenciados (mensajes del usuario vs bot)
- Tablas de horarios y asignaturas con barras visuales de importancia/dificultad
- Spinner animado mientras Gemini procesa
- Markdown renderizado en los mensajes del bot (tablas, negritas, listas)

---

9. Comandos disponibles en el chat

Comando	Acción
/perfil	Editar nombre, asignaturas, horas y preferencias
/horario	Pedir un horario semanal completo optimizado
/resumen	Ver tabla visual de asignaturas y disponibilidad

Comando	Acción
/reset	Reiniciar historial de chat (mantiene el perfil)
/ayuda	Mostrar lista de comandos
/salir	Cerrar el chatbot

10. Pasos para ejecutarlo

Requisitos previos

- **Python 3.10 o superior** instalado
  → Descarga: https://www.python.org/downloads/
- **API Key de Google Gemini** gratuita
  → Obtener: https://aistudio.google.com/app/apikey

Paso 1 — Abrir la terminal en el directorio del proyecto

powershell

cd C:\Users\MSI\.gemini\antigravity\scratch\study-chatbot

Paso 2 — (Opcional pero recomendado) Crear entorno virtual

powershell

python -m venv .venv

.\.venv\Scripts\activate

Paso 3 — Instalar dependencias

powershell

pip install -r requirements.txt

Nota: Si omites este paso, el propio script instalará las dependencias automáticamente la primera vez que lo ejecutes.

Paso 4 — Configurar la API Key (elige una opción)

Opción A — Variable de entorno (recomendado):

powershell

$env:GEMINI_API_KEY = "tu_clave_aqui"

Opción B — Archivo .env:
Crea un archivo .env en la carpeta del proyecto con:

GEMINI_API_KEY="tu_clave_aqui"

Opción C — Introducirla cuando el script la solicite:
Simplemente ejecuta el script y sigue las instrucciones en pantalla.

Paso 5 — Ejecutar el chatbot

powershell

python chatbot.py

Flujo de primera ejecución

1. El script detecta que no hay perfil guardado
2. Te guía para configurar tu nombre y nivel de estudios
3. Introduces tus asignaturas con su importancia y dificultad
4. Configuras las horas disponibles cada día de la semana
5. Indicas si quieres priorizar el ocio en fin de semana
6. El perfil se guarda y el chatbot arranca con un mensaje de bienvenida personalizado

Ejemplos de preguntas que puedes hacer

¿Cuántas horas debería dedicar a cada asignatura esta semana?

Tengo el examen de Matemáticas en 3 días, ¿cómo reorganizo mi tiempo?

Dame un plan de choque para la semana de exámenes.

¿Qué técnica de estudio me recomiendas para Historia?

Genera un horario para esta semana con bloques Pomodoro.

¿Cómo distribuyo el fin de semana si tengo una salida el sábado por la tarde?